lux.nvim Architectural Rewrite Plan

Problem Statement

rocks.nvim was a Neovim plugin manager that wrapped the `luarocks` CLI for package management. We are rewriting it from scratch to use **lux** (https://github.com/lumen-oss/lux) instead — specifically the `lux-lua` Rust crate which exposes lux's operations as Lua bindings via mlua.

The current commit has nuked the old codebase. The Lua module has been renamed to `lux-nvim`. The only hard requirements are: 1. Keep `require("rocks.api")` working as a compatibility shim for existing extensions 2. Everything else can be fully restructured

---

Key Decisions (resolved via clarifying questions)

Decision	Choice
`require("rocks.api")` compat	✅ Keep via shim → `require("lux-nvim.api")`
User config format	Switch to `lux.toml` in nvim config dir (breaking)
Operations gap in lux-lua	Contribute install/uninstall/update/sync to lux-lua
User plugin declaration	`~/.config/nvim/lux.toml` (user's config dir as lux project)
Extension system	Keep as `LuxHandler` (renamed from `RockHandler`)
RTP integration	Use lux `nvim_mode` config flag (lux lays out tree like `site/pack`), then `packadd`
lux-lua Rust additions	Plan describes required Rust additions in detail

---

Architecture Overview

The new lux.nvim treats the user's Neovim config directory as a **lux project**. The user declares their plugins in `~/.config/nvim/lux.toml` under `[dependencies]`. lux.nvim reads this project via `lux.project.new()`, uses the lux-lua API for all state queries, and will use new lux-lua operations (to be contributed) for install/uninstall/update/sync.

lux's `nvim_mode` flag causes packages to be laid out in the traditional `site/pack/lux/opt/<name>` structure, meaning `packadd` continues to work as before and colorschemes/autoload scripts remain compatible.

File Layout

```
lua/
├── rocks/
│   └── api.lua             # Backwards-compat shim → require("lux-nvim.api")
│
└── lux-nvim/
    ├── init.lua            # Public entry point; setup() + packadd shim
    ├── api.lua             # Public extension API (equiv of old rocks.api)
    │
    ├── config/
    │   ├── init.lua        # User-facing config module (vim.g.lux_nvim)
    │   ├── check.lua       # Config validation (validate LuxConfig table)
    │   └── internal.lua    # Resolved internal config; builds lux-lua Config
object
    │
    ├── lux.lua             # lux-lua Lua bindings wrapper + full type
annotations
    │
    ├── state.lua           # State queries: installed, outdated, removable
packages
    ├── cache.lua           # Async lazy cache layer (populated on startup)
    │
    ├── operations/
    │   ├── init.lua        # Re-exports: add, prune, sync, update, pin, unpin
    │   ├── add.lua         # Install / update-to-version packages
    │   ├── prune.lua       # Remove packages and their orphaned deps
    │   ├── sync.lua        # Sync lux.toml [dependencies] ↔ installed state
    │   ├── update.lua      # Update all/specific packages to latest
    │   ├── pin.lua         # Pin a package to its current version
    │   ├── unpin.lua       # Unpin a package
    │   └── handlers.lua    # LuxHandler extension registry
    │
    ├── runtime.lua         # Source start plugins; packadd helpers
    ├── commands.lua        # :Lux user command + subcommand registration
    ├── health.lua          # :checkhealth lux-nvim
    ├── fzy.lua             # Fuzzy filter adapter (wraps fzy rock)
    └── log.lua             # Structured logging (wraps fallo or similar)

plugin/
└── lux-nvim.lua            # Neovim plugin entrypoint (guards, init sequence)
```

Required lux-lua Rust Additions

The current `lux-lua` crate only exposes `config`, `project`, `loader`, and `operations.search`. The following additions are required before the full Lua implementation can be completed.

1. `operations.rs` — New Operations

These need to be added to the `operations(lua)` table in `lux-lua/src/operations.rs`:

```rust
use lux_lib::operations::{Install, PackageInstallSpec, Uninstall, Sync, Update};
use lux_lib::lockfile::LocalPackageId;

// --- Install ---
// lux.operations.install(specs, config) -> async -> nil | err
// specs: list of { name: string, version_req: string?, pin: bool?, opt: bool? }
table.set(
    "install",
    lua.create_async_function(
        |_, (specs, config): (Vec<PackageInstallSpecLua>, ConfigLua)| async move
{
            let _rt = lua_runtime().enter();
            let lua_version = config.0.lua_version().into_lua_err()?;
            let tree = config.0.user_tree(lua_version).into_lua_err()?;
            let package_db = RemotePackageDB::from_config(
                &config.0, &Progress::<ProgressBar>::no_progress()
            ).await.into_lua_err()?;
            Install::new(&config.0)
                .tree(tree)
                .package_db(Some(package_db))
                .packages(specs.into_iter().map(|s| s.0).collect())
                .install().await.into_lua_err()
        },
    )?,
)?;

// --- Uninstall ---
// lux.operations.uninstall(package_ids, config) -> async -> nil | err
table.set(
    "uninstall",
    lua.create_async_function(
        |_, (ids, config): (Vec<LocalPackageIdLua>, ConfigLua)| async move {
            let _rt = lua_runtime().enter();
            Uninstall::new(&config.0)
                .packages(ids.into_iter().map(|id| id.0))
                .uninstall().await.into_lua_err()
        },
    )?,
)?;

// --- Sync ---
// lux.operations.sync(project, config) -> async -> nil | err
table.set(
    "sync",
    lua.create_async_function(
        |_, (project, config): (ProjectLua, ConfigLua)| async move {
            let _rt = lua_runtime().enter();
            Sync::new(&project.0, &config.0)
                .sync_dependencies().await.into_lua_err()
```

```
        },
    )?,
)?;

// --- Update ---
// lux.operations.update(package_reqs | nil, config) -> async -> nil | err
// Pass nil to update all packages
table.set(
    "update",
    lua.create_async_function(
        |_, (reqs, config): (Option<Vec<PackageReqLua>>, ConfigLua)| async move {
            let _rt = lua_runtime().enter();
            Update::new(&config.0)
                .packages(reqs.map(|rs| rs.into_iter().map(|r| r.0).collect()))
                .update().await.into_lua_err()
        },
    )?,
)?;

// --- Outdated ---
// lux.operations.outdated(config) -> async -> table<name, { installed: string,
latest: string }>
table.set(
    "outdated",
    lua.create_async_function(
        |_, config: ConfigLua| async move {
            let _rt = lua_runtime().enter();
            get_outdated_packages(&config.0).await.into_lua_err()
        },
    )?,
)?;
```

2. `config.rs` — nvim_mode builder method

Add to `ConfigBuilderLua` in `lux-lua/src/lua_impls.rs` (and wire through `lux-lib`):

```
// Enables Neovim-compatible tree layout:
// packages are placed in site/pack/lux/opt/<name>/ instead of
// the content-addressable .lux/ layout.
methods.add_method("nvim_mode", |_, this, enabled: Option<bool>| {
    Ok(ConfigBuilderLua(this.0.clone().nvim_mode(enabled)))
});
```

The corresponding `nvim_mode` field in `lux-lib/src/config/mod.rs` needs to be wired to the `RockLayoutConfig::new_nvim_layout()` entrypoint layout.

3. `PackageInstallSpecLua` — new wrapper type

Add to `lux-lua/src/lua_impls.rs`:

```
#[derive(Debug, Clone)]
pub struct PackageInstallSpecLua(pub PackageInstallSpec);
```

```rust
impl FromLua for PackageInstallSpecLua {
    fn from_lua(value: LuaValue, lua: &Lua) -> LuaResult<Self> {
        // Accept either a PackageReq string ("name version_req")
        // or a table { name, version?, pin?, opt? }
        match value {
            LuaValue::Table(tbl) => {
                let name: String = tbl.get("name")?;
                let version_req: Option<String> = tbl.get("version")?;
                let pin: Option<bool> = tbl.get("pin")?;
                let opt: Option<bool> = tbl.get("opt")?;
                // build PackageInstallSpec from these fields...
            }
            _ => {
                let req = PackageReqLua::from_lua(value, lua)?;
                Ok(PackageInstallSpecLua(PackageInstallSpec::from(req.0)))
            }
        }
    }
}
```

Lua Type Definitions

All types are defined as LuaCATS (`---@class`) annotations, kept in `lux.lua` for reference.

```lua
--- A package spec from the user's lux.toml [dependencies]
---@class LuxSpec
---@field name string
---@field version? string  Version constraint: "*", "~> 8.0", "== 1.0.0"
---@field opt? boolean     If true, not loaded eagerly (maps to opt pack)
---@field pin? boolean     If true, ignored by :Lux update

--- An installed package from the lux tree lockfile
---@class LuxPackage
---@field name string
---@field version string
---@field id string         Content-addressable ID (hash prefix)
---@field pinned boolean
---@field opt boolean

--- An outdated installed package
---@class OutdatedLuxPackage: LuxPackage
---@field target_version string

--- Callback signature for LuxHandler operations
---@alias lux_handler_callback fun(
---   on_progress: fun(message: string),
---   on_error: fun(message: string),
---   on_success?: fun(opts: lux_handler.on_success.Opts)
```

```
--- )

---@class lux_handler.on_success.Opts
---@field action 'install' | 'prune'
---@field package LuxPackage
---@field dependencies? string[]

--- Extension handler (replaces RockHandler)
---@class LuxHandler
---@field get_sync_callback? fun(spec: LuxSpec): lux_handler_callback | nil
---@field get_prune_callback? fun(specs: table<string, LuxSpec>):
lux_handler_callback | nil
---@field get_install_callback? fun(toml_ref: MutLuxTomlRef, arg_list: string[]):
lux_handler_callback | nil
---@field get_update_callbacks? fun(toml_ref: MutLuxTomlRef):
lux_handler_callback[]

--- Mutable reference to the lux.toml dependency table (for handlers)
---@class MutLuxTomlRef
---@field dependencies? table<string, LuxSpec | string>
---@field [string] unknown  Extra fields added by handlers

--- User-facing configuration (vim.g.lux_nvim)
---@class LuxNvimConfig
---@field lazy? boolean           Lazily populate search cache (default: false)
---@field dynamic_rtp? boolean    Auto-add new packages to rtp after install
(default: true)
---@field generate_help_pages? boolean  Re-generate helptags after install
(default: true)
---@field update_remote_plugins? boolean  Run :UpdateRemotePlugins after install
(default: true)
---@field auto_sync? boolean | 'prompt' | 'disable'  Sync on startup if out-of-
date (default: false)
---@field config_path? string     Path to lux.toml (default: stdpath("config")/
lux.toml)
---@field extra_servers? string[] Additional package servers
```

Module-by-Module Design

`plugin/lux-nvim.lua` (Neovim plugin entrypoint)

```
1. Guard: if vim.g.loaded_lux_nvim then return end
2. Check minimum Neovim version (>= 0.10)
3. Require config.internal → resolves vim.g.lux_nvim into LuxConfig + lux-lua
Config
4. Call lux.loader() to install the lux package loader
5. Append lux bin dir (config:user_tree():bin()) to vim.env.PATH
6. Create :Lux user command via commands.create_commands()
7. Unless config.lazy: spawn async task to populate all caches
```

8. Source start plugins: read project toml [dependencies], call runtime.source_start_plugins()
9. Set vim.g.loaded_lux_nvim = true

lux-nvim/config/internal.lua

Responsibilities: - Read `vim.g.lux_nvim` (or call it if it's a function) - Validate with `config.check.validate()` - Resolve `config_path` to the lux.toml path (default: `stdpath("config")/lux.toml`) - Build the lux-lua `Config` object: `lua local lux = require("lux") local config = lux.config.builder() :user_tree(data_dir) -- stdpath("data")/lux :lua_version("5.1") :extra_servers(extra_servers) :nvim_mode(true) -- Neovim-compatible pack layout :build()` - Expose `internal.lux_config` (the lux-lua Config userdata) - Expose `internal.project` (the lux-lua Project userdata for user's nvim config dir) - Expose `internal.get_user_deps()` ⟶ reads project toml [dependencies] ⟶ `table<name, LuxSpec>`

lux-nvim/lux.lua (lux-lua bindings wrapper)

A thin wrapper around `require("lux")` that: - Provides full LuaCATS type annotations for all lux-lua userdata types - Initializes the lux-lua module once and caches it - Exposes a clean API surface:

```lua
local lux_m = require("lux")

local lux = {}

-- lux.config.builder():...:build() → ConfigLua userdata
lux.config = lux_m.config

-- lux.project.new(path) → ProjectLua | nil
-- lux.project.current() → ProjectLua | nil
lux.project = lux_m.project

-- lux.operations.search(query, config) → async → table<name, string[]>
-- lux.operations.install(specs, config) → async
-- lux.operations.uninstall(ids, config) → async
-- lux.operations.sync(project, config) → async
-- lux.operations.update(reqs?, config) → async
-- lux.operations.outdated(config) → async → table<name, OutdatedLuxPackage>
lux.operations = lux_m.operations

-- lux.loader() → installs custom package loader
lux.loader = lux_m.loader

return lux
```

lux-nvim/state.lua

All state queries use the lux-lua Tree/Lockfile API. No CLI subprocess calls.

```lua
---@type async fun(): table<string, LuxPackage>
state.installed_packages = nio.create(function()
    local config = require("lux-nvim.config.internal")
    local tree = config.lux_config:user_tree("5.1")
    local lockfile = tree:lockfile()
    local rocks = lockfile:rocks()  -- table<id_string, LocalPackageLua>
    -- Convert LocalPackageLua → LuxPackage table
    local result = {}
    for id, pkg in pairs(rocks) do
        result[pkg:name()] = {
            name    = pkg:name(),
            version = pkg:version(),
            id      = id,
            pinned  = pkg:pinned(),
            opt     = -- read from lux.toml spec
        }
    end
    return result
end)

---@type async fun(): table<string, OutdatedLuxPackage>
state.outdated_packages = nio.create(function()
    local config = require("lux-nvim.config.internal")
    local lux = require("lux-nvim.lux")
    -- Uses new lux.operations.outdated() contributed to lux-lua
    return lux.operations.outdated(config.lux_config)
end)

---@type async fun(): string[]
state.query_removable_packages = nio.create(function()
    local installed = state.installed_packages()
    -- Walk lockfile: collect all dependency IDs
    local config = require("lux-nvim.config.internal")
    local tree = config.lux_config:user_tree("5.1")
    local lockfile = tree:lockfile()
    local dependent_ids = {}
    for _, pkg in pairs(lockfile:rocks()) do
        for _, dep_id in ipairs(pkg:dependencies()) do
            dependent_ids[dep_id] = true
        end
    end
    -- Filter: packages not depended on by anything and not in user's lux.toml
    local user_deps = config.get_user_deps()
    return vim.iter(pairs(installed))
        :filter(function(name, pkg)
            return not dependent_ids[pkg.id]
                and not user_deps[name]
                and name ~= "lux.nvim"
        end)
```

```lua
        :map(function(name, _) return name end)
        :totable()
end)

---@type async fun(user_deps?: table<string, LuxSpec>): SyncStatus
state.out_of_sync_packages = nio.create(function(user_deps)
    -- Compare user's lux.toml [dependencies] with installed lockfile
    -- Returns { to_install, to_updowngrade, to_prune, external_actions }
    ...
end)
```

lux-nvim/cache.lua

Same structure as old `cache.lua`, but backed by lux-lua APIs.

```lua
local _packages_cache = nil        -- table<name, string[]> from
lux.operations.search
local _outdated_cache = nil        -- table<name, OutdatedLuxPackage>
local _removable_cache = nil       -- string[]

-- Fires autocmds:
--   User LuxCachePopulated          (data = packages table)
--   User LuxOutdatedCachePopulated  (data = outdated table)
--   User LuxRemovableCachePopulated (data = removable list)

cache.populate_packages_cache      -- async: calls lux.operations.search("*",
config)
cache.populate_outdated_cache      -- async: calls
lux.operations.outdated(config)
cache.populate_removable_cache     -- async: calls
state.query_removable_packages()

cache.try_get_packages()           -- → table | empty (triggers populate if nil)
cache.try_get_outdated()           -- → table | empty
cache.try_get_removable()          -- → string[] | empty

cache.invalidate_removable()       -- after install/prune
cache.invalidate_outdated()        -- after update
```

lux-nvim/operations/add.lua

```lua
--- Install a package and add it to lux.toml [dependencies]
---@param arg_list string[]  First element is package name, optional version
follows
---@param opts? { skip_prompts?: boolean, cmd?: 'install'|'update', callback?:
fun(pkg: LuxPackage) }
add.add = function(arg_list, opts)
    -- 1. Parse arg_list → name, version_req, extra opts (opt=true, pin=true)
    -- 2. Show progress via fidget
    -- 3. Check handlers.get_install_handler_callback(name, arg_list)
    --    - If handler: delegate to handler callback
    --    - Else: use lux.operations.install([{name, version_req, opt, pin}],
```

```
lux_config)
    -- 4. On success:
    --    - Update lux.toml via project:add({ regular = {PackageReq} },
lux_config)
    --      OR write to toml directly using pathlib/toml-edit approach
    --    - If dynamic_rtp: add package to rtp via runtime.packadd()
    --    - If generate_help_pages: vim.cmd.helptags()
    --    - If update_remote_plugins: vim.cmd.UpdateRemotePlugins()
    --    - Invalidate caches
    --    - Call opts.callback if provided
end
```

Note: `project:add()` in lux-lua both modifies the TOML *and* installs. We should decide whether to use it directly (letting lux manage toml writing) or to install manually then update toml separately for finer progress control. The recommended approach is to use `project:add()` directly since it handles dependency resolution atomically.

lux-nvim/operations/prune.lua

```
prune.prune = function(package_name, opts)
    -- 1. Show progress via fidget
    -- 2. Check handlers.get_prune_handler_callback()
    -- 3. Use lux.operations.uninstall([id], lux_config)
    --    - Get LocalPackageId from tree:match_rocks(PackageReq)
    -- 4. Remove from lux.toml via project:remove({ regular = {PackageName} },
lux_config)
    --    OR use project:remove() which handles both
    -- 5. Prune orphaned dependencies (lux may handle this automatically)
    -- 6. Invalidate caches
end
```

lux-nvim/operations/sync.lua

```
sync.sync = function(user_deps, on_complete)
    -- 1. If user_deps nil, read from config.get_user_deps()
    -- 2. Compute out-of-sync state via state.out_of_sync_packages(user_deps)
    -- 3. Show progress via fidget
    -- 4. Process external_actions (LuxHandler callbacks) in parallel
    -- 5. Install to_install packages: lux.operations.install(specs, lux_config)
    -- 6. Up/downgrade to_updowngrade: lux.operations.install(specs, lux_config)
with force
    -- 7. Prune to_prune: lux.operations.uninstall(ids, lux_config)
    --    - OR use lux.operations.sync(project, config) if project-level sync
handles all
    -- 8. Invalidate all caches
    -- 9. Call on_complete if provided
end
```

Preferred: Use `lux.operations.sync(project, lux_config)` directly for steps 5-7, as it handles resolution atomically. External actions (LuxHandler) are processed separately before/after.

lux-nvim/operations/update.lua

```lua
update.update = function(package_name, opts)
    -- If package_name provided: update single package
    --   lux.operations.update({PackageReq(name, "*")}, lux_config)
    -- Else: update all non-pinned packages
    --   lux.operations.update(nil, lux_config)
    -- Also run get_update_callbacks() from registered LuxHandlers
    -- Invalidate outdated cache
end
```

lux-nvim/runtime.lua

The `nvim_mode` config flag causes lux to install packages into `<user_tree>/pack/lux/opt/<name>/`. This means `packadd <name>` works natively.

```lua
--- Source all non-opt start plugins from lux.toml [dependencies]
---@param user_deps table<string, LuxSpec>
function runtime.source_start_plugins(user_deps)
    local not_found = {}
    for name, spec in pairs(user_deps) do
        if not spec.opt and spec.version then
            local ok = pcall(vim.cmd.packadd, name)
            if not ok then
                table.insert(not_found, name)
            end
        end
    end
    -- Handle not_found: auto_sync or prompt user
    if #not_found > 0 then
        -- ... same prompt logic as before
    end
end

--- Load a single optional package
---@param name string
---@param opts? { bang?: boolean }
function runtime.packadd(name, opts)
    local ok, err = pcall(vim.cmd.packadd, { name, bang = opts and opts.bang })
    if not ok then
        log.warn(("Package %s not found: %s"):format(name, err))
        return false
    end
    return true
end
```

lux-nvim/commands.lua

Identical structure to old `commands.lua` but updated for new module names and types:

- `:Lux install {name} {version?} {opt=?} {pin=?}` ⟶ `operations.add()`
- `:Lux prune {name}` ⟶ `operations.prune()`

- `:Lux sync` ⟶ `operations.sync()`
- `:Lux update {name?}` ⟶ `operations.update()`
- `:Lux edit` ⟶ opens `lux.toml` (from `config.config_path`)
- `:Lux pin {name}` ⟶ `operations.pin()`
- `:Lux unpin {name}` ⟶ `operations.unpin()`
- `:Lux log` ⟶ `log.open_logfile()`

Subcommand registration API (`commands.register_subcommand()`) remains identical.

`lux-nvim/api.lua` (public extension API)

This replaces `rocks.api`. All function signatures are preserved but backed by lux-lua:

```lua
-- These are the SAME function signatures as old rocks.api,
-- allowing extensions to switch from require("rocks.api") to
-- require("lux-nvim.api") with zero code changes (beyond the require path).

api.try_get_cached_packages()
    -- → table<name, string[]>  (search results; lux.operations.search)

api.try_get_cached_outdated_packages()
    -- → table<name, OutdatedLuxPackage>

api.query_lux_packages(callback)
    -- Async: queries remote, passes table<name, string[]> to callback
    -- Replaces api.query_luarocks_rocks()

api.query_luarocks_rocks(callback)
    -- Deprecated alias for api.query_lux_packages() for backwards compat

api.fuzzy_filter_package_tbl(tbl, query, opts)
    -- → filtered table (unchanged, uses fzy.lua)

api.query_installed_packages(callback)
    -- Async: passes table<name, LuxPackage> to callback

api.get_lux_toml_path()
    -- → string (path to user's lux.toml)

api.get_rocks_toml_path()
    -- Deprecated alias for api.get_lux_toml_path()

api.get_lux_toml()
    -- → parsed lux.toml as Lua table (project:toml() converted to table)

api.get_rocks_toml()
    -- Deprecated alias for api.get_lux_toml()

api.get_user_packages()
    -- → table<name, LuxSpec>
```

```lua
api.get_user_rocks()
    -- Deprecated alias for api.get_user_packages()

api.register_lux_subcommand(name, cmd)
    -- Registers a :Lux subcommand

api.register_rocks_subcommand(name, cmd)
    -- Deprecated alias for api.register_lux_subcommand()

api.register_lux_handler(handler)
    -- Registers a LuxHandler for custom install/prune/update logic

api.register_rock_handler(handler)
    -- Deprecated alias for api.register_lux_handler()

api.install(rock_name, version, opts)
    -- Invokes :Lux install programmatically
```

`lua/rocks/api.lua` (compatibility shim)

```lua
-- rocks.api compatibility shim
-- Allows extensions that use require("rocks.api") to continue working
-- without any code changes.
vim.deprecate(
    "require('rocks.api')",
    "require('lux-nvim.api')",
    "4.0.0",
    "lux.nvim"
)
return require("lux-nvim.api")
```

API Mapping Table (Old → New)

Old (luarocks-based)	New (lux-lua-based)
`luarocks.cli({"install", name})`	`lux.operations.install([{name, version_req}], config)`
`luarocks.cli({"remove", name})`	`lux.operations.uninstall([LocalPackageId], config)`
`luarocks.cli({"list", "--porcelain"})`	`config:user_tree("5.1"):lockfile():rocks()`
`luarocks.cli({"list", "--outdated"})`	`lux.operations.outdated(config)` *(new)*
`luarocks.search_all(cb, opts)`	`lux.operations.search(query, config)`
`state.installed_rocks()`	lockfile `:rocks()` → convert to LuxPackage table
`state.outdated_rocks()`	`lux.operations.outdated(config)` *(new)*
`state.query_removable_rocks()`	walk lockfile deps graph (same logic, no CLI)
`state.out_of_sync_rocks(user_rocks)`	`state.out_of_sync_packages(user_deps)` (same logic)
`adapter.init_site_symlinks()`	not needed — lux `nvim_mode` handles layout
`adapter.create_symlink(rock)`	not needed — lux `nvim_mode` handles layout
`runtime.packadd(rock_spec)`	`runtime.packadd(name, opts)` (same, simpler)
`runtime.source_start_plugins(user_rocks)`	`runtime.source_start_plugins(user_deps)` (same logic)
`config.rocks_path`	`config.lux_config:user_tree("5.1"):root()`
`config.config_path` (rocks.toml)	`config.config_path` (lux.toml path)
`config.get_user_rocks()`	`config.get_user_deps()` (from project toml)
`config.get_servers()`	`config.lux_config:extra_servers()`
`operations.add(arg_list, opts)`	`operations.add(arg_list, opts)` (same surface)
`operations.prune(name)`	`operations.prune(name)` (same surface)
`operations.sync(user_rocks, cb)`	`operations.sync(user_deps, cb)` (same surface)
`operations.update(name?, opts)`	`operations.update(name?, opts)` (same surface)
`operations.pin(name)` / `unpin(name)`	same surface, backed by lockfile mutations
`cache.try_get_rocks()`	`cache.try_get_packages()`
`cache.try_get_outdated_rocks()`	`cache.try_get_outdated()`
`cache.try_get_removable_rocks()`	`cache.try_get_removable()`

Startup Sequence (plugin/lux-nvim.lua)

```
1.  Guard: vim.g.loaded_lux_nvim → early return
2.  Check nvim >= 0.10
3.  Load log module
4.  Load config.internal → build lux-lua Config with nvim_mode=true
5.  Enable lux loader: lux.loader()
6.  Create :Lux command: commands.create_commands()
7.  Append tree bin dir to PATH: vim.env.PATH
8.  Unless config.lazy:
      nio.run(cache.populate_packages_cache)
      nio.run(cache.populate_outdated_cache)
      nio.run(cache.populate_removable_cache)
9.  Read user_deps from project toml (lux.toml [dependencies])
10. source_start_plugins(user_deps) — synchronous, ensures proper init order
11. vim.g.loaded_lux_nvim = true
```

Implementation Order

1. **Rust (lux-lua contributions)**:
   - ☐ Add `nvim_mode` to `ConfigBuilder` and wire to `RockLayoutConfig::new_nvim_layout()` in lux-lib
   - ☐ Add `nvim_mode(bool?)` builder method to `ConfigBuilderLua` in lux-lua
   - ☐ Add `PackageInstallSpecLua` wrapper type in lux-lua
   - ☐ Add `lux.operations.install()` async function in lux-lua
   - ☐ Add `lux.operations.uninstall()` async function in lux-lua
   - ☐ Add `lux.operations.sync()` async function in lux-lua
   - ☐ Add `lux.operations.update()` async function in lux-lua
   - ☐ Add `lux.operations.outdated()` async function in lux-lua
2. **Lua (lux.nvim core)**:
   - ☐ `log.lua` — structured logging
   - ☐ `fzy.lua` — fuzzy filter adapter (copy from old, update requires)
   - ☐ `config/check.lua` — config validation
   - ☐ `config/init.lua` — user-facing config
   - ☐ `config/internal.lua` — resolved config + lux-lua Config build
   - ☐ `lux.lua` — lux-lua bindings wrapper with annotations
   - ☐ `state.lua` — all state queries via lux-lua tree/lockfile
   - ☐ `cache.lua` — async cache layer
   - ☐ `operations/handlers.lua` — LuxHandler registry
   - ☐ `operations/add.lua`
   - ☐ `operations/prune.lua`
   - ☐ `operations/sync.lua`
   - ☐ `operations/update.lua`
   - ☐ `operations/pin.lua` + `unpin.lua`

- ☐ `operations/init.lua` — re-exports
- ☐ `runtime.lua`
- ☐ `commands.lua`
- ☐ `health.lua`
- ☐ `init.lua` (public API)
- ☐ `api.lua` (extension API)
- ☐ `lua/rocks/api.lua` (compat shim)
- ☐ `plugin/lux-nvim.lua` (entrypoint)

Open Questions / Notes

- **Pin/unpin storage**: The old `rocks.toml` stored `pin = true` in the file. With lux.toml's `[dependencies]` format, pinned state needs to live either in the lockfile (`PinnedState`) or as a custom toml annotation. Recommended: store `pin = true` as a TOML inline table extension and read it back on sync. The lux lockfile already has `PinnedState` — we should persist pin decisions to the lockfile via `lockfile:map_then_flush()`.
- **opt= handling**: Similarly, `opt = true` in lux.toml dependencies may need a custom extension field OR we can track it in a separate metadata section. This needs coordination with the lux project to agree on a convention.
- **lux.toml creation**: On first run, if no `lux.toml` exists in the nvim config dir, we should create one with a default template (analogous to the old `DEFAULT_CONFIG`). The new template should use lux.toml format with `[dependencies]`.
- **dev rocks**: The old system supported `version = "dev"` / `"scm"`. With lux, dev packages use `dev = true` in the config builder. We need to translate `version = "dev"` in lux.toml to use the dev servers.
- **rocksCmd → LuxCmd**: The `RocksCmd` type should be renamed to `LuxCmd` in the new API, with a deprecated alias.